

CS 677: Distributed and Operating Systems

Lab 3: Design Document

Manan Parikh (mananrajeshb@umass.edu), Noah Walls (nwalls@umass.edu)

Overview:

Background: Previously in lab 2, we created a multi-threaded containerized stock market application made up of microservices, each one deployed as an HTTP server that supports communication via REST API.

Goal: Add frontend stock caching, Replication of order service, and a new order lookup functionality. Introduce fault-tolerance in the case of order service crashes. Deploy all services on AWS and access remotely.

Previous Interface:

Request (Client → Frontend)	Description	Response
POST /orders {name, quantity, type}	Place order at frontend (order)	JSON{transaction_number error object}
GET /stocks/<stock_name>	Request stock info at frontend (lookup)	JSON{stock_name, price, quantity error object}

New Frontend REST API: To support order lookup (aka order query) requests, the frontend now accepts HTTP GET requests at its pre-existing /orders endpoint, which clients access with a request in the form of:

GET /orders/<order_number> (order number means transaction number)

While making its requests, the client stores all successful trade requests in a HashMap, where:

Key = transaction number returned by the server (order number that is stored in orders.csv file).

Value = an ArrayList of [stock name, quantity, operation type (buy/sell)].

Once all requests are complete, the client begins its order-checking phase where it sends an order lookup request for each successful trade in its HashMap to retrieve the order details from the server. The server response is parsed into [name, quantity, type] and compared with the locally stored trade data for that transaction number. If there's a mismatch, it prints out the difference; otherwise, it prints that the server response matches the local record. If there are no records found for the requested order number, frontend service will send a JSON response with a top level error object containing two fields; code and message.

Client Configuration: Clients will send a mix of trade and stock lookup requests. A lookup request is made using GET /stocks/<stock_name>, and based on the returned quantity and a random chance (defined by the probability parameter), a trade request using POST /orders/ is made.

The total number of requests (lookup + trade) is controlled using the total_requests parameter, which can be passed as an argument inside the Dockerfile.client and is picked up by Client.java.

The arguments accepted by the client are as follows:

args[0]: Probability – a double between 0 and 1 to control how often a trade follows a lookup (default = 0.8).

args[1]: Requests - Total number of requests to make (default = 8).

args[2]: PrintLatencyInfo - A boolean flag to optionally print latency info (default = false). If true, the client will print latency for each request, and at the end, it will also provide average latency for lookup and trade requests.

args[3]: FrontendAddress - IPv4 address of the frontend service. This is optional when all clients and services are running locally, but required when deploying the services on an EC2 instance while clients run locally.

Part 1: Caching

Goal: Implement LRU cache in frontend to store stock data and reduce repeated calls to the catalog service. Allow catalog service to inform frontend when a stock's information has changed so the cache may be updated.

Cache: The cache is implemented using a combination of a HashMap for O(1) lookup and a doubly linked list to maintain the access order. Each cache entry is represented by a Node object containing the stock name as the key and a HashMap<String, Object> as the value, storing either valid data or an error message. This structure was already being used by the frontend service to package data before responding to client lookup requests.

Replacement Policy: The most recently accessed node is always moved to the front (head) of the list, while the least recently used node resides near the tail. When a new entry is added and the cache exceeds its predefined capacity (5 entries), the node at the tail is removed, effectively evicting the least recently used item. The put() method adds or updates entries and maintains the correct order in the linked list, while the get() method returns cached values and also promotes the node to the most recently used position (head).

Cache Consistency: When the order-service sends a request to the catalog-service to validate whether a trade request can be completed or not, at the time, if catalog service decides yes, it will send an invalidation request to

the frontend service to the /updateCache endpoint to remove the entry of a stock that is being traded from the cache to avoid sending stale results to the client.

When the catalog-service sends an invalidation request to the frontend service to remove a stock entry for the stock that is currently being traded, Cache.java has write lock implemented on the cache. This ensures that when one thread acquires the lock for that stock, any other thread attempting a cache lookup for the same stock must wait until the first thread releases the write lock.

Configuration: To turn off caching, we need to pass "false" as an argument to the frontend service via the frontend Dockerfile. The last line in this file declares the entry point for this service, and the last argument is "false", which indicates to the frontend service not to use the caching. Dockerfile.frontendCachingDisabled provides an example of how to do this. To customize the cache size, provide two arguments when starting the frontend service: the first should be "true" to enable caching, and the second should specify the desired cache size as a string (e.g., "10"). If no arguments are provided, caching is enabled by default with a size of 5.

Part 2: Replication

Goal: Implement three replicas of the order service (one leader and two followers) and to enforce update propagation from the leader to the followers. Additionally, we wanted these to be true replicas, built from the same application code and Dockerfile, not just three different order services serving the same purpose.

Docker-Compose: In docker-compose.yml, we replaced our *orderservice* service with services *orderservice0*, 1, and 2, all built from the same app code and Dockerfile. This Dockerfile exposes container port 8082 which we map to 8082, 8083, 8084 respectively in compose for each order service replica, giving them each a unique host machine port. Environment variables contain the hardcoded base URL of each container.

Leader Assignment: ElectOrderServiceLeader() in the frontend is run on server start and whenever an order request fails to reach the leader order service replica. This function will iterate down the list of possible replica IDs, pinging the /health endpoint of each replica with a GET request until it gets a 200 code response. It returns the base URL of the new leader if successful, which will be the replica with the highest ID that is online. The frontend stores this value and uses it as the address to forward order requests to. On the order service side, replicas can determine whether they are the leader or a follower by the way they receive orders.

When a replica receives an order via the /trade endpoint, it indicates that the replica is a leader, as the frontend service forwards its order requests to the leader replica (by POST request at /trade). The replica sets its isLeader boolean to true whenever an order is received at this endpoint.

When a replica receives an order via the /update endpoint, it indicates that it is a follower, as the leader replica makes POST requests here to propagate its updates to the followers. The replica sets isLeader to false when it receives an order here.

Update Propagation: Immediately after updating its own order log with a successful transaction, the leader will call UpdateFollowers(requestBody). This requestBody is the string containing the body of the request that the frontend sent to the order service via POST at /trade. UpdateFollowers() maps this requestBody to an orderRequest object and POSTs it to /update at each of the follower replicas, both of which update their order log.

Part 3: Fault Tolerance

Goal: Handle request rerouting in the event of an order replica crash and support resynchronization of order log data upon replica recovery, ensuring the application is crash tolerant and minimizing impact on client.

Leader Crash Handling: Leader crash detection takes place inside the Order Handler of the frontend service, which is where order related requests are sent to by the client and subsequently forwarded to the order service leader. In the event that the leader order replica has crashed, an order / order lookup request forwarded by the frontend will initially fail, as the specified host is unreachable. When the connection fails, an exception is thrown, triggering a leader re-election in the front end. The request forwarding process takes place inside a try block within a while loop. This loop ensures that the request can be re-submitted with a new leader in the case that the original leader was unresponsive.

To minimize the delay experienced by the client as a result of a crash, a timeout of 30 ms is enforced for the URL connection between the frontend and order service, preventing hanging connections. Additionally, timeouts varying from 20 to 40 ms are enforced for every URL connection throughout

```
Frontend Order Handler Leader Crash Detection
(both trade and query requests)

ReadClientRequest();
While(leaderCrash) {
    try {
        response = ForwardToOrderService(leader);
        // ^ exception thrown here
    } catch {
        leaderCrash = true;
        leader = ElectOrderServiceLeader();
        continue; -- jump back to top of loop
    }

    leaderCrash = false;
    ReplyToClient(response);
}
```

Figure 1

the program. Any URL connection made in the application (not the client) will be running on the same local machine, meaning delays greater than 10 ms likely indicate that a service is down. Enforcing short timeouts prevents bottlenecks from forming, ensuring that failures are not noticeable to the client.

Crash Recovery: On server start, all replicas are followers until the first request comes in where the frontend selects a leader. After a leader crashes, it will come back as a follower. When starting up, a replica will attempt to recover orders from the other two replicas by calling `recoverOrders()`, which finds the latest transaction number stored in its order log and sends a GET request to the `/inform` endpoint of one of the replicas. This query takes the form of: `<replica_base_URL>/inform?transactionNumber=<latest_transaction_number>`

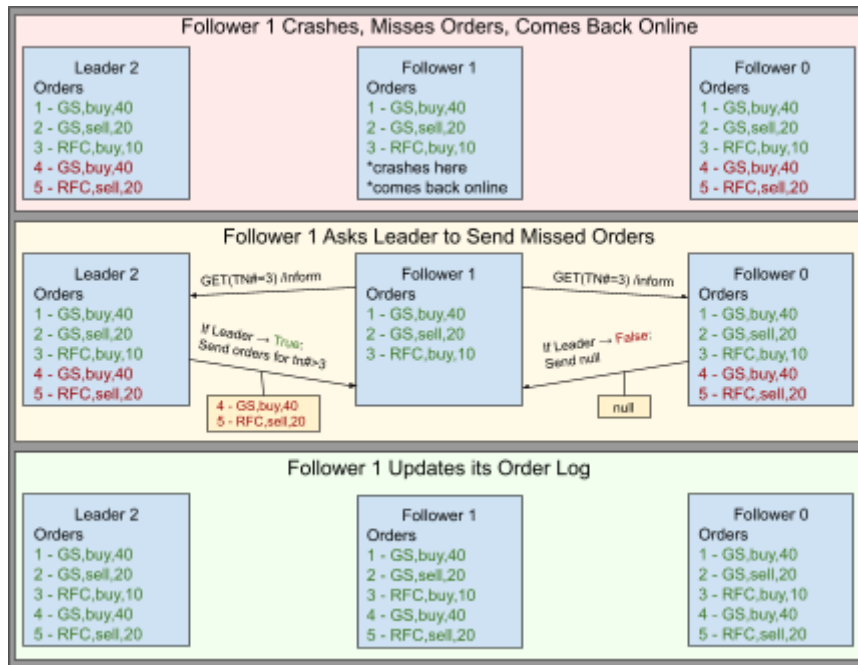


Figure 2

If it receives order log updates, it will not attempt to get updates from the other replica. Upon receiving the updates, it will write them to its log.

On the leader side, it listens for these `/inform` requests at its Inform Handler. Upon receiving the request, it first checks if it's the leader and then checks the transaction number against its own log and returns all subsequent orders as a list in JSON format. The replica will only respond with orders if it is the leader, the transaction number is valid, and it has more recent orders. Otherwise it will return null. Additionally, a transaction number of -1 indicates that the requester replica has no orders stored, and the leader should return all of its orders.

Part 4: Testing and Evaluation with Deployment on AWS

Goal: Deploy microservices on AWS, allowing clients to connect and make requests from any machine remotely. Conduct testing to measure the performance impact of caching as well as the difference in response times between trade and lookup requests with varying order probabilities applied. Determine the impact of simulated replica crashes on the server response time as well as data consistency between order log files.

Deployment: The entire application (without clients) is deployed on an EC2 t2.medium instance (more details about configurations in the README instructions). To deploy, we start an instance, SSH in (or connect with AWS GUI), install docker/docker-compose, copy all necessary files and folders from our project folder to the instance via SCP, and build/start the entire app within the instance. Requests can be made on the local machine either via web browser or by building/starting clients. This approach allows our app to be deployed on AWS without changing application code or docker related files (besides modifications required for testing).

Testing: To facilitate the required performance testing, we configured a dedicated Docker Compose setup using the `docker-compose-client.yml` file. This configuration defines a service named `clientforloadtesting`, responsible for simulating multiple client instances to generate load and measure latency.

The service is set up to deploy 5 replicas of the client container, which are built using the `Dockerfile.client`. This Dockerfile gets modified to include the public IP address of our EC2 instance, ensuring that the client containers can communicate with the deployed service endpoints during testing. Additionally, the required probability parameter (used in request generation logic) is set within this Dockerfile to match the intended testing scenario.

To initiate the load testing, we executed the following command:

```
docker-compose -f 'docker-compose-client.yml' up -d --build 'clientforloadtesting'
```

This command builds and launches the client containers in detached mode. Each container runs an instance of the `Client.java` application, which is responsible for issuing requests to the server, measuring the latency for each request, and finally calculating and reporting the average latency over all executed requests.

References

- [HTTP Response Status Codes](#)
- [Building Containers Within EC2 \(Instead of Using ECR\)](#)
- [AWS EC2 Security Groups](#)
- [LRU Cache Implementation](#)
- [LRU Cache](#)
- [Deserialization \(com.fasterxml.jackson.databind.type.TypeFactory\)](#)
- [Timeout settings for URL connections \(java.net.URLConnection\)](#)